

RESEARCH

Open Access



Max–Min semantic chunking of documents for RAG application

Csaba Kiss^{1*}, Marcell Nagy¹ and Péter Szilágyi²

*Correspondence:

Csaba Kiss

kisscs@math.bme.hu

¹Department of Stochastics,
Institute of Mathematics, Budapest
University of Technology and
Economics, Műegyetem rkp. 3.,
Budapest 1111, Hungary

²Nokia Bell Labs, Budapest, Hungary

Abstract

Retrieval-augmented generation (RAG) systems have emerged as a powerful approach to enhance large language model (LLM) outputs, however, their effectiveness heavily depends on document chunking strategies. Current methods, often arbitrary or size-based segmentation, fail to preserve semantic coherence, leading to suboptimal retrieval and reduced output quality. To overcome this limitation, we introduce Max–Min semantic chunking, a novel method utilizing semantic similarity and a Max–Min algorithm to identify semantically coherent text. We evaluated our approach on three distinct datasets, assessing clustering efficiency via adjusted mutual information (AMI) and generation coherence through accuracy on a RAG-based multiple-choice question answering test. Across the datasets, Max–Min semantic chunking achieved superior performance with average AMI scores of 0.85, 0.90, and an average accuracy of 0.56 (averaged across LLMs). This significantly outperformed the next best method, the Llama Semantic Splitter (AMI: 0.68, 0.70; accuracy: 0.53). The improvements in the AMI scores were statistically significant.

Keywords Natural language processing (NLP), Retrieval augmented generation (RAG), Document and text processing, Large language models (LLMs), Semantic chunking

1 Introduction

The growing complexity of text generation tasks in modern natural language processing (NLP) applications, which most commonly use retrieval-augmented generation (RAG) [1], has increased the need for efficient document segmentation techniques. These RAG systems originally utilized encoder-decoder seq2seq transformers or their adaptations, such as those presented in [2]. As large language models (LLMs) became more prevalent, they emerged as the primary generators in modern RAGs. However, managing irrelevant context became more significant, as it can distract these models [3, 4]. Traditional methods for document chunking often fail to capture nuanced semantic relationships, limiting their ability to organize information in a way that enhances retrieval and subsequent text generation tasks [5]. These limitations become especially evident in systems where the quality of document segmentation directly affects the relevance and accuracy of generated responses, such as in RAG applications where both retrieval precision and chunk coherence are critical for model performance.



© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

To address this challenge, numerous chunking methods have been proposed. These methods typically employ pre-defined sentence or paragraph boundaries, window-based techniques, and syntactic indicators for chunking [6–8]. However, such approaches frequently face difficulties in ensuring the semantic coherence of the resulting paragraphs. Further details on these methods are provided in Sect. 2.

In this paper, we propose a novel Max–Min semantic chunking algorithm to address this issue. Our approach frames the chunking task as a (dynamic) clustering problem, where the goal is to group consecutive sentences based on their semantic similarity while respecting the linear structure of the document. This method relies on the similarity between sentence embeddings to dynamically form semantically coherent chunks, ensuring that critical information is not lost during segmentation. By employing a Max–Min similarity strategy, the algorithm adaptively determines chunk boundaries based on both the highest and lowest (hence the name “Max–Min”) semantic similarities within a set of consecutive sentences. By adopting this approach, we avoid the need for arbitrary chunk size selection and the complex process of hyperparameter tuning, as described in ref. [9]. The schematic workflow of our chunking algorithm is illustrated in Fig. 1. The source code for this work is available at <https://github.com/hsdslab/MaxMinChunking>.

We evaluate our approach on two fronts: first, by using two datasets and treating the task as a dynamic clustering problem where the ground-truth labels are available, and comparing it against existing chunking methods using well-established clustering metrics. Second, by integrating the chunking method into a RAG application and assessing its impact on the accuracy of responses generated by three LLMs on a multiple-choice test. The results indicate that the Max–Min chunker consistently and statistically significantly outperforms existing methods across three datasets, delivering higher-quality chunks that improve both in-chunk coherence and retrieval performance.

2 Related work

Traditional chunking strategies focus only on the structure of a text, overlooking its meaning. The simplest approach splits text into fixed-sized chunks, typically based on a predefined word (e.g., 100-word chunks, [1, 10]) or token count (e.g., 288-token chunks, [11]). This can be done with non-overlapping chunks, but this method often breaks up sentences, leading to loss of meaning. To address this, overlapping chunking is commonly used [12], adding a step parameter (also measured in tokens and is smaller than the chunk length) that allows chunks to partially overlap. However, this approach still creates redundant chunks containing fragmented sentences, complicating retrieval

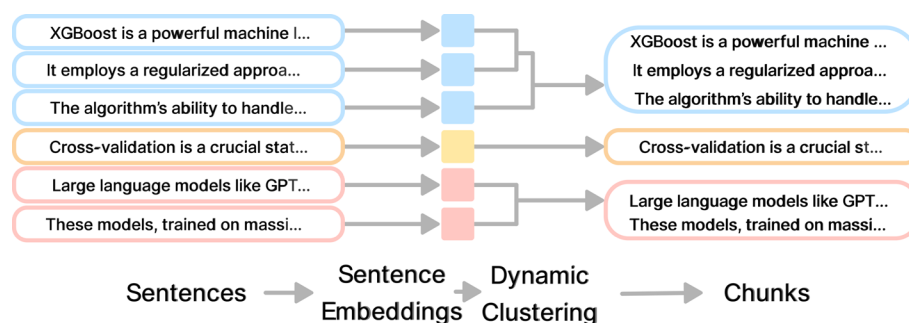


Fig. 1 The workflow of the Max–Min semantic chunking algorithm. The colors represent the topics (XGBoost, cross-validation, and LLMs)

in RAG applications and reducing the likelihood of selecting the most relevant chunk. Chunks can be also created by assigning a fixed number of sentences to each chunk. This approach maintains the integrity of sentences but does not consider the semantic coherence of the text, potentially dividing meaningful sections across multiple chunks.

Another chunking strategy involves using the document's inherent structure by dividing it into titles, headings, paragraphs, and sections [13]. This approach relies on the author to produce coherent chunks. However, some documents may lack a clear structure, or sections and paragraphs may vary significantly in length, which can hinder the retrieval algorithm's ability to provide the most relevant text to the LLM. Additionally, this method can lead to chunks that exceed the context window of the LLM model. To address the limitations of document structure-based chunking, the recursive chunking method was introduced in ref. [14]. This method also leverages the document's structure and can be applied in two ways. When paragraphs are too large, they are recursively split into smaller parts until each chunk is below the desired token count. Conversely, if paragraphs are too short, they can be recursively merged until they meet the target size, resulting in chunks closer to the ideal length for inclusion in an LLM prompt.

On the other hand, chunking methods based on the document's structure, such as element-based chunking [8], segment documents using inherent structures like titles, tables, and paragraphs. This approach is particularly useful in specialized applications, such as financial report analysis, where the document's structure is key to both retrieval and generation tasks [8]. By utilizing these structural elements, element-based chunking improves retrieval accuracy and generation relevance in RAG applications, outperforming simpler methods like fixed-size or sentence-based chunking. However, this approach is highly domain-specific and difficult to generalize.

Furthermore, these methods still ignore the text's semantics, as they rely instead on token counts or the document's structural layout, which causes a problem with those paragraphs that discuss different topics. To address this, a popular semantic chunking approach was developed that is based on the idea of G. Kamradt [15]. This semantic chunker is implemented in most frameworks (e.g., LlamaIndex and LangChain) which uses the vectors of text embedding methods. Most implemented versions use a fixed-sized sentence window approach [7, 16], setting a sentence window size (e.g., 3 sentences) and comparing consecutive sentence groups by a similarity metric in the embedding space. By sliding the window one step at a time, they evaluate possible breakpoints between sentences, deciding where to split based on a threshold or percentile. These approaches produce semantically coherent chunks; however, we hypothesize that a fixed window size may not be optimal for determining the best segmentation points in the document (e.g. cannot handle parenthetical or footnote sentences). Moreover, recently in ref. [17] Qu et al. questioned whether semantic chunking justifies its computational overhead compared to fixed-size chunking. Our results demonstrate significant performance gains over the existing semantic chunker while simultaneously reducing computational overhead. Additionally, we note that since RAG systems inherently require sentence embeddings for retrieval, these already-computed embeddings can be utilized either through aggregation or hierarchical chunk-sentence representations, with the only additional computational cost being the dynamic clustering step, which is relatively lightweight.

Large language models (LLMs) have also been explored as document segmenters [18, 19]. This approach gained traction after the release of ChatGPT in November 2022, when there was a surge in interest in the capabilities of LLMs. Since then, these models have been applied to almost anything, thanks to their user-friendly nature, which enables even those without deep technical skills to tackle various tasks. However, in document segmentation, while LLMs can be effective for smaller documents, they still require some pre-segmentation due to the limited context window of the models. Moreover, this method demands substantial computational power, making it both expensive and slow. Due to these limitations, LLM-based chunking is not an ideal or cost-effective solution for creating a RAG application with large documents.

Recently, a promising chunking-free method was introduced in ref. [5], called Chunking-Free In-Context Retrieval (CFIC). This approach has demonstrated strong results in applications like open-domain question answering and large-scale search engines. CFIC eliminates the need for manual chunking by utilizing transformer hidden states and specialized decoding strategies to retrieve relevant text evidence from long documents directly. Although CFIC excels in open-domain tasks like search engine queries, it is less suited for static documents such as structured RAG applications, since the retrieved texts have to fit in the model's context to be processed. In these cases, large, static, and well-defined documents must be chunked before processing due to the model's context size limitations, making CFIC more appropriate for smaller text sources, such as webpages.

3 Methodology

We approach chunking as a dynamic/temporal clustering problem, where consecutive sentences are grouped into chunks based on their embeddings. Unlike traditional clustering methods, our approach respects the “temporal nature” of sentences within a document, meaning that the sentences within a cluster are consecutive. To accommodate this, we developed a method that considers both the embedding of each sentence and its position.

The algorithm processes the document sentence by sentence. For each sentence, we decide whether to add it to the current chunk (i.e., cluster) or start a new one. First, using a text embedding model, we embed all sentences into a high-dimensional space. Let $s_1, s_2, \dots, s_n$ denote the sentences in the document, which consists of n sentences. Moreover, let us suppose that when the algorithm reaches the k th sentence (denoted by s_k), it has already grouped the previous few (e.g. five) sentences $\{s_{k-5}, s_{k-4}, \dots, s_{k-1}\}$ into the current chunk, denoted by C . At this point (i.e., when reaching s_k), the question is whether s_k should be included in chunk C or it should create a new chunk.

To make this decision, we calculate the minimum pairwise cosine similarity between the embeddings of sentences within the current chunk C :

$$\min_sim(C) = \min_{s,t \in C} sim_{cos}(s, t) = \min_{s,t \in C} \frac{E(s)^T E(t)}{\|E(s)\| \cdot \|E(t)\|}, \quad (1)$$

where $E(s)$ represents the embedding of sentence y . This equation helps us identify the least similar pair of sentences within the chunk, providing a measure of how closely related the sentences are to each other, which helps us decide whether the new sentence is similar enough to the sentences of the chunk. The idea is that the new sentence s_k

could be added to C if the similarity of s_k to any of the sentences of chunk C is not (significantly) lower than $\min_sim(C)$.

Hence, as a next step, we compute the maximum cosine similarity between the new sentence s_k and the sentences of the current chunk C :

$$\max_sim(s_k, C) = \max_{s \in C} \text{sim}_{\cos}(s_k, s) = \max_{s \in C} \frac{E(s_k)^T E(s)}{\|E(s_k)\| \cdot \|E(s)\|} \quad (2)$$

This equation captures the “closeness” of s_k and C , namely, the highest similarity of the new sentence s_k with the existing chunk C , indicating how well s_k fits with the sentences already grouped.

The key idea is that if the minimum similarity within the chunk C is less than the maximum similarity between s_k and any sentence in the chunk, then we should add s_k to the chunk, i.e., formally:

$$\text{if } \min_sim(C) < \max_sim(s_k, C) \text{ then add } s_k \text{ to chunk } C. \quad (3)$$

To relax the condition, when the cardinality of C is small, we allow $\max_sim(s_k, C)$ to be lower than the minimum similarity. To do so, we introduce a threshold to ensure that the similarity between the new sentence and the chunk exceeds a certain level before inclusion. This threshold $\text{thr}(C)$ depends on C and is computed as follows:

$$\text{thr}(C) = \max\{c \cdot \min_sim(C) \cdot \sigma(|C|), \text{hard_thr}\} \quad (4)$$

where c is a hyperparameter, σ is the sigmoid function, and $|C|$ is the cardinality of chunk C , i.e., the number of sentences currently in the chunk. This formulation makes it increasingly difficult for new sentences to be added as the chunk grows, while the *hard_thr* prevents the algorithm from adding loosely related sentences to the chunk if $\min_sim(C)$ becomes too small.

If $\max_sim(s_k, C)$ exceeds this threshold, we add s_k to chunk C and move on to the next sentence, s_{k+1} , repeating the process.

An initialization issue occurs when the current chunk consists of only one sentence, i.e., $C = \{s_{k-1}\}$. In this case, we compare the cosine similarity between the embeddings of s_{k-1} and s_k , i.e., $\text{sim}_{\cos}(s_{k-1}, s_k)$ to the *hard_thr* adjusted with a constant *init_const*. If $\text{init_const} \cdot \text{sim}_{\cos}(s_{k-1}, s_k) > \text{hard_thr}$, we add s_k to the chunk, otherwise, s_k starts a new chunk. The pseudo-code of the Max–Min semantic chunking is detailed in Algorithm 1.

```

1: Input: A set of sentences  $S = \{s_1, s_2, \dots, s_n\}$ , a text embedding model  $E$ , and
   hyperparameters of  $c$ ,  $hard\_thr$ , and  $init\_const$ 
2: Output: Chunked sentences, i.e., a partition of  $S$ .
3: Initialize an empty chunk  $C = \{\}$ 
4: for each sentence  $s_k$  in document do
5:   if  $C$  is empty then
6:     Start a new chunk with  $s_k$ 
7:      $C \leftarrow \{s_k\}$ 
8:   else if  $C$  contains one sentence  $\{s_{k-1}\}$  then
9:     Compute cosine similarity:

```

$$\text{sim}(s_{k-1}, s_k) \leftarrow \frac{E(s_{k-1})^T E(s_k)}{\|E(s_{k-1})\| \cdot \|E(s_k)\|}$$

```

10:    if  $init\_const \cdot \text{sim}(s_{k-1}, s_k) \geq hard\_thr$  then
11:      Add  $s_k$  to  $C$ 
12:    else
13:      Start a new chunk with  $s_k$ 
14:    end if
15:  else
16:    Compute  $\text{min\_sim}(C)$  of the current chunk  $C$ :

```

$$\text{min_sim}(C) \leftarrow \min_{s, t \in C} \frac{E(s)^T E(t)}{\|E(s)\| \cdot \|E(t)\|}$$

```

17:    Compute  $\text{max\_sim}(s_k, C)$  between  $s_k$  and the current chunk  $C$ :

```

$$\text{max_sim}(s_k, C) \leftarrow \max_{s \in C} \frac{E(s_k)^T E(s)}{\|E(s_k)\| \cdot \|E(s)\|}$$

```

18:    Compute threshold:

```

$$\text{thr}(C) \leftarrow \max\{c \cdot \text{min_sim}(C) \cdot \sigma(|C|), hard_thr\}$$

```

19:    if  $\text{max\_sim}(s_k, C) \geq thr$  then
20:      Add  $s_k$  to  $C$ 
21:    else
22:      Start a new chunk with  $s_k$ 
23:    end if
24:  end if
25: end for
26: Return: Chunked sentences

```

Algorithm 1 Max–Min semantic chunking algorithm

4 Data and experimental setup

Both the Max–Min and LlamaIndex semantic splitters require an embedding model. For this purpose, we have opted for the BGE-base-en v1.5 model from HuggingFace [20]. This model transforms text into a 768-dimensional vector space.

Measuring the effectiveness of RAG applications is challenging due to the complexity of assessing how well the retrieved information supports generated responses. Factors such as chunk quality, relevance to the questions, and variability in language model outputs complicate evaluation. To compare the performance of our algorithm with other chunking methods, we conducted two evaluation approaches (evaluation as clustering

and in a RAG application) aimed at systematically assessing the performance of different chunking methods while addressing these challenges.

We utilize three distinct datasets for evaluation: two datasets for measuring the clustering efficiency of our method, and a dataset of multiple-choice questions to test the performance (accuracy) of a RAG application, which was built on documents from the 3GPP telecommunications standards.

The first two datasets were designed to assess our chunker's efficiency from a clustering point of view. We scraped Wikipedia pages about machine learning (ML) and data science-related topics for the first dataset (some examples can be seen in Fig. 1), and collected famous people's pages for the second dataset, extracted their paragraphs, and shuffled them. We chose these topics to include technical and non-technical texts in our analysis. The reason why we only scraped pages for a dataset from one area is to make the task non-trivial (since it would be easier to solve if completely different topics were shuffled). Each paragraph was assigned a unique ID, which was inherited by all its sentences. This ID served as the ground truth for clustering evaluation. The goal was to identify the start and end of paragraphs using these known clustering labels for each sentence. Specifically, for the ML-related dataset, we scraped 500 Wikipedia pages, obtaining 3930 paragraphs and 20,906 sentences, with a median of 5 sentences per paragraph. Furthermore, for the dataset that contains paragraphs about famous people, we collected 100 Wikipedia pages with 6200 paragraphs, 36,982 sentences, with a median of 5 sentences per paragraph. After shuffling the paragraphs, we sequentially labeled the sentences, starting with label 0 for the first paragraph's sentences, 1 for the second's, and so on. The algorithms were then provided with the list of sentences and tasked with chunking them to get back the original paragraphs.

The second approach is based on a RAG system, where an LLM has to answer multiple-choice test questions, and we compare the percentage of correct answers using different chunking strategies. For building this RAG application, we used the technical telecommunication specifications and documents of the 3GPP, specifically the 23 series for technical realization (Stage 2) for 3 G and beyond, the 25 series for radio aspects in 3 G and beyond, the 36 series for LTE, LTE-Advanced, and LTE-Advanced Pro radio technology, and the 38 series for radio technology beyond LTE. For each chunking method, we set up a corresponding RAG application, ensuring that the only difference among them was how the same document was split into chunks. All setups used the same ColBERT framework established in ref. [21] (i.e. maximum similarity) to retrieve the 10 most relevant chunks for a given query and the same three LLM models to answer 100 multiple-choice questions based on the 3GPP specifications acquired from ref. [22], with questions categorized as easy, intermediate, and hard. For each question, four possible answers are provided, with only one being correct. To ensure reproducibility and rule out randomness, the temperature of the models was set to zero. We selected three language models of varying sizes: the small-scale Mistral 8B, the medium-scale LLaMA 3.2 70B, and the large-scale GPT-3.5 Turbo.

4.1 Clustering evaluation

The first evaluation approach treats the task as a clustering problem on the shuffled paragraphs from various Wikipedia pages, as described in Sect. 4. Here, we compare the LlamaIndex semantic chunker with our Max-Min semantic chunker, using fixed sentence

count paragraphs as a baseline. The algorithms were given sentence lists and tasked with reconstructing these into paragraphs. We evaluated their performance using the top five widely used clustering metrics available in the scikit-learn Python package [23] that account for ground truth labels: adjusted mutual information (AMI) score [24], normalized mutual information (NMI) score (also referred to as V-measure by [25]), adjusted rand (AR) index [26], completeness score (CS) [25], and homogeneity score (HS) [25].

As discussed in Sect. 2, the LlamaIndex method relies on two hyperparameters: buffer size and percentile threshold. For both chunking methods, we optimized these hyperparameters — the two for LlamaIndex and three for our Max–Min method (as described in Sect. 3) using a five-fold cross-validation. We aimed to maximize the average of the five clustering metrics on the training set through the hyperparameters using the Nelder–Mead algorithm, introduced in [27], then we used the optimal hyperparameters to evaluate the methods on the test set across the five-fold data splits. The hyperparameter search space for the Max–Min method was $(0, 1) \times (0, 2) \times (0, 2)$ for *hard_thr*, *c*, and *init_const*, while for the LlamaIndex semantic method, it was $\{1, 2, 3, 4, 5\} \times (50, 99)$ for buffer size and percentile threshold. For the baseline, we calculated the median sentence count per paragraph in the training set during cross-validation and applied this to segment the test set. We compared both methods against this fixed-size chunking approach, as other chunking strategies rely on prior knowledge of document structure, which is not available in this case. We then averaged the results across the five cross-validation test sets and performed t-tests to assess the differences between chunking methods.

4.2 RAG evaluation

The second evaluation method examines the effectiveness of our chunking approach in a RAG application. We compared our Max–Min semantic chunker with three common chunking techniques:

- *Llama Sentence Splitter*: Uses document structure to segment text, as outlined in [6] and detailed in Sect. 2.
- *MaxSize Grouping*: A recursive chunking technique based on document structure, introduced in [8] and detailed in Section 2.
- *Llama Semantic Splitter*: A semantic splitter based on a fixed window size [7].

As a baseline, we tested the language model’s performance on the questions without any context to observe its standalone capabilities.

Our RAG system, based on 3GPP technical documents, employed different chunking methods while keeping other components consistent. Employing the ColBERT framework [21], we retrieved the top 10 chunks per query, which were incorporated into the prompts of the three LLMs to address the 100 multiple-choice questions, as described in Sect. 4.

The prompt used for the LLM model was as follows:

Table 1 Performance of the chunking methods according to the clustering metrics. Bold values represent the best scores for each dataset and metric

	Chunking method	AMI	NMI	AR	CS	HS
ML dataset	Fixed Size	0.70	0.93	0.56	0.92	0.93
	Llama Semantic	0.68	0.92	0.54	0.91	0.93
	Max–Min	0.85	0.96	0.72	0.95	0.98
Famous People dataset	Fixed Size	0.71	0.93	0.56	0.91	0.94
	Llama Semantic	0.70	0.92	0.56	0.92	0.93
	Max–Min	0.90	0.98	0.82	0.97	0.99

Table 2 Paired t-test p-values comparing method performance across metrics

	Metric	Max–Min vs Llama	Max–Min vs Fixed Size	Llama vs Fixed Size
ML dataset	AMI	< 0.001***	< 0.001***	< 0.001***
	NMI	< 0.001***	< 0.001***	< 0.001***
	AR	< 0.001***	< 0.001***	< 0.001***
	CS	< 0.001***	< 0.001***	0.049*
	HS	< 0.001***	< 0.001***	0.034*
Famous People dataset	AMI	< 0.001***	< 0.001***	0.011*
	NMI	< 0.001***	< 0.001***	< 0.001***
	AR	< 0.001***	< 0.001***	0.886
	CS	< 0.001***	< 0.001***	0.995
	HS	< 0.001***	< 0.001***	< 0.001***

Significance levels: * $p < 0.05$; ** $p < 0.01$; *** $p < 0.001$

Please provide the answers to the following multiple choice question: {question}

Considering the following context: {context}
Options: {options}
Write only the option number corresponding to the correct answer.

This evaluation framework allowed a direct comparison of the accuracy of question-answering across different chunking methods, with retrieval and response generation remaining consistent.

5 Results

5.1 Clustering evaluation results

In the clustering-based evaluation, we compared our Max–Min method to the well-known LlamaIndex Semantic Splitter, and to a baseline non-semantic sentence window-based chunker. The comparison results and the corresponding p -values (calculated via a one-sided Student's t -test) are presented in Table 1 and Table 2, respectively. As the tables suggest, the Max–Min method consistently outperformed the LlamaIndex Semantic Splitter across all calculated clustering metrics and for both clustering datasets. This indicates that our approach is superior at identifying semantically coherent paragraphs and detecting precise changepoints where the text transitions to a new topic. Notably, the LlamaIndex Semantic Splitter performed slightly worse than the baseline fixed-sized (fixed number of sentences per paragraph) method, whereas our Max–Min chunker surpassed both. This supports the critique in [17] where the authors questioned whether the SemanticSplitter offers meaningful improvements over fixed-sized methods.

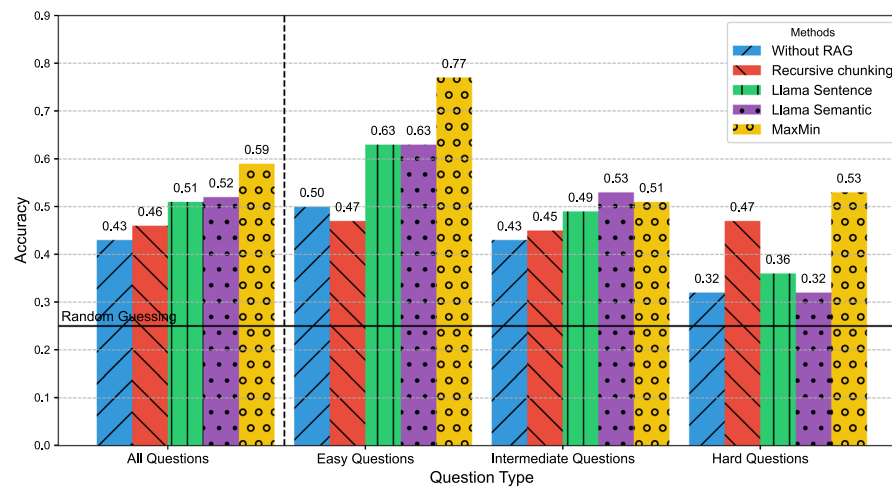


Fig. 2 Comparison of different chunking methods for answering questions in a RAG application with GPT–3.5 Turbo

Table 3 Performance (accuracy) of different document chunking methods in a RAG application, evaluated using three large language models (GPT–3.5, Llama 3.2 70B, Mistral 8B) across all questions and by difficulty categories (Easy, Intermediate, Hard). Bold values represent the best scores for each LLM and question type

	All questions	Easy questions	Intermediate questions	Hard questions
GPT-3.5				
Without RAG	0.43	0.5	0.43	0.32
Recursive Chunking	0.46	0.47	0.45	0.47
Llama Sentence	0.51	0.63	0.49	0.37
Llama Semantic	0.52	0.63	0.53	0.32
Max–Min	0.59	0.77	0.51	0.52
Llama 3.2 70B				
Without RAG	0.46	0.5	0.47	0.37
Recursive Chunking	0.58	0.67	0.59	0.42
Llama Sentence	0.55	0.6	0.59	0.37
Llama Semantic	0.58	0.7	0.61	0.32
Max–Min	0.57	0.6	0.61	0.42
Mistral 8B				
Without RAG	0.36	0.4	0.37	0.26
Recursive Chunking	0.48	0.57	0.47	0.37
Llama Sentence	0.51	0.63	0.49	0.37
Llama Semantic	0.49	0.63	0.45	0.37
Max–Min	0.51	0.7	0.45	0.37

5.2 RAG evaluation results

The question-answering accuracy of the LLM using the splitting strategies described in Subsection 4.2 is shown in Fig. 2 and detailed in Table 3. With four possible answers for each question, and only one being correct, the expected accuracy from random guessing is 25%.

Utilizing the large-scale GPT–3.5 Turbo, the Max–Min chunker outperforms all the methods tested across the question set. While the LlamaIndex Semantic Splitter shows a slight advantage on intermediate-level questions, the Max–Min chunker demonstrates significantly better performance on both easy and hard questions. Notably, on the hard questions, the LlamaIndex approach performs comparably to context-free LLM querying

(marginally better than random guessing), further supporting the critique in ref. [17] and confirming that the Max–Min chunker is the most effective method in this experiment. When employing medium- or small-scale LLMs, the advantage of the Max–Min method diminishes slightly but remains comparable or superior to other methods. This could be attributed to the high complexity of the questions, which often demand both contextual understanding and calculations. Smaller LLMs, despite the provided context, tend to resort to guessing. This is particularly noticeable in the performance of the small-scale LLM on Hard questions, where all methods barely exceed random guessing.

5.3 Hyperparameter sensitivity analysis

We assessed hyperparameter sensitivity using the clustering dataset composed of machine learning-related Wikipedia paragraphs. A grid search was performed to evaluate different hyperparameter settings, using the average of five clustering metrics (detailed in Sect. 4.1). Figure 3 visualizes these results through three pairwise heatmaps colored by the clustering performance. The analysis reveals that the Max–Min chunker method is most sensitive to the `hard_thr` parameter. This parameter defines the minimum cosine similarity required for a sentence to join a cluster, and performance drops significantly when `hard_thr` falls below 0.4. Above this threshold – a range encompassing more realistic and reasonable values – the method exhibits stable and robust performance. The `init_const` parameter also impacts performance, though to a lesser extent than `hard_thr`; performance declines if `init_const` is below 0.8. This parameter influences the initial merging likelihood of the first two sentences: higher values increase this probability, leading to improved clustering. This observation is logical,

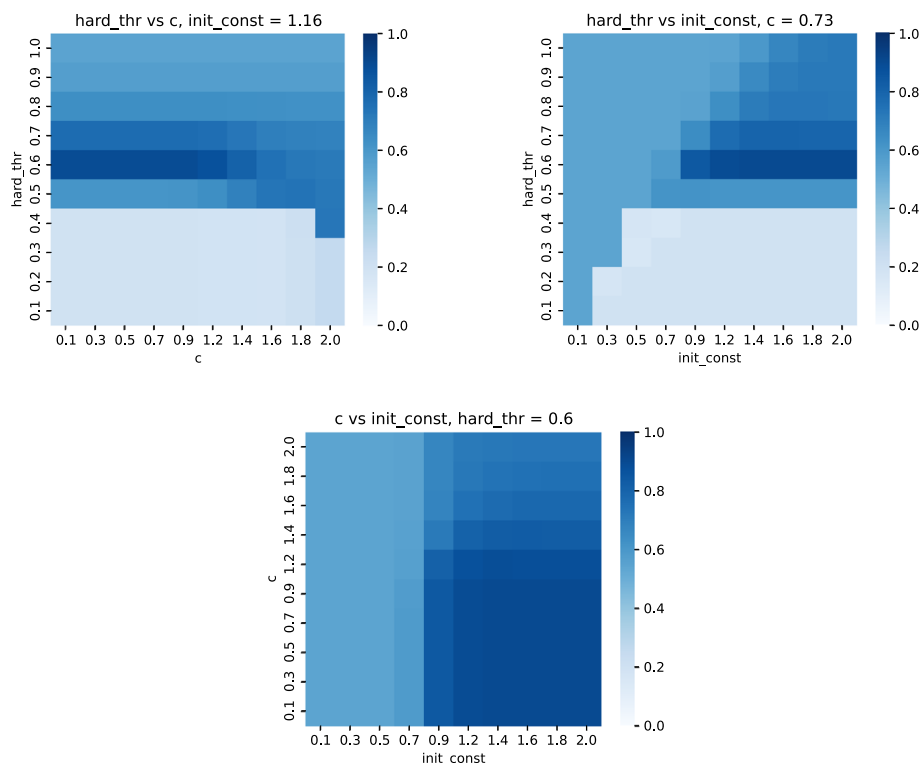


Fig. 3 Average clustering score, computed via the mean of the five scores, for the Max–Min method on the Machine Learning related clustering dataset with different hyperparameter settings

Table 4 Running times of the chunking methods broken down into embedding and chunking steps. The numbers show the average and the standard deviation of 10 runs

Chunking method	Embedding	Chunking	Total
LlamaIndex Sentence	–	1.26s $\pm$ 6.12ms	1.26s $\pm$ 6.12ms
LlamaIndex Semantic	79.0s $\pm$ 64.8ms	6.65s $\pm$ 13.9ms	85.65s $\pm$ 66.27ms
Max–Min	24.7s $\pm$ 22.4ms	7.61s $\pm$ 9.74ms	32.31s $\pm$ 24.43ms

Table 5 Memory usage of the chunking methods broken down into embedding and chunking steps

Chunking method	Embedding (peak GPU memory usage)	Chunking (peak CPU memory usage)
LlamaIndex sentence	–	30.02 MB
LlamaIndex semantic	320.58 MB	2.31 MB
Max–Min	161.54 MB	1.67 MB

as an individual sentence rarely forms a semantically complete and isolated chunk on its own. In contrast, the method demonstrates less sensitivity to the parameter c . However, the heatmaps in Fig. 3 suggest that lower values (e.g., $c < 1$) can yield slight performance improvements.

Based on this grid-search analysis and cross-validation hyperparameter optimization, we recommend the following initial hyperparameter settings: `hard_thr` = 0.6, `init_const` = 1.5, and c = 0.9.

5.4 Comparison of running times and memory usage

We also evaluated the computational performance of our Max–Min chunker against the LlamaIndex Semantic Splitter, focusing on running times and memory consumption. All evaluations used the ML-related Wikipedia dataset and the BGE-base-en-v.15 embedding model (detailed in Sect. 4).

Table 4 details the running times, separating them into embedding and chunking phases. The LlamaIndex Semantic Splitter embeds fixed-size overlapping chunks (buffer size set to 4 in our experiments), while our Max–Min chunker embeds individual sentences. Consequently, LlamaIndex exhibits longer embedding times due to processing larger text windows. Conversely, the Max–Min method incurs slightly more time in the chunking phase for its cosine similarity computations. However, Max–Min demonstrates superior overall speed because its efficient sentence-level embedding outweighs the computational cost of its subsequent chunking logic. For baseline comparison, the table also includes the LlamaIndex Sentence Splitter, which operates without embeddings.

As Table 5 demonstrates, our Max–Min approach also proved more memory-efficient. It consumed less GPU memory during text embedding, primarily due to processing shorter text segments (individual sentences). Furthermore, its post-embedding CPU memory requirements were lower than those of LlamaIndex.

The embeddings were processed on an NVIDIA GeForce RTX 3060 GPU (12 GB VRAM). All other computations were performed on a 4-core Intel Core i5-6500 CPU @ 3.20GHz with 62 GB RAM.

6 Conclusion

In this study, we introduced Max–Min Semantic Chunking, a novel method for document segmentation in Retrieval-Augmented Generation (RAG) systems. Our approach employs a Max–Min strategy to dynamically group sentences based on semantic similarity, producing coherent and contextually meaningful chunks. We rigorously evaluated the method in two settings: in a clustering task and as part of a RAG application. The results demonstrate that our method consistently outperforms existing chunking techniques, such as the widely used LlamaIndex Semantic Splitter, in terms of clustering quality, question-answering RAG performance (especially on difficult questions), and computational efficiency.

Our clustering evaluations revealed that the Max–Min chunker significantly outperformed fixed-size and LlamaIndex semantic chunking methods across all metrics (Adjusted Mutual Information, Normalized Mutual Information, Adjusted Rand Index, Completeness, and Homogeneity). This underscores its enhanced ability to group semantically related sentences.

In the RAG experiment, the Max–Min Semantic Chunker significantly outperformed other methods when using a large-scale LLM, especially on hard questions where other chunking methods often failed to retrieve relevant context. For instance, on such questions, the LlamaIndex Semantic Splitter offered minimal improvement over querying the LLM without any retrieved context. Although performance was comparable with medium- or small-scale LLMs, these findings highlight the critical advantage of dynamically adapting chunk boundaries to textual semantics. Furthermore, our method proved faster overall than the LlamaIndex Semantic Splitter, as its sentence-level embedding efficiency compensated for the additional cosine similarity computations during chunking.

Despite these promising results, this study has limitations. First, our clustering datasets, though designed to be challenging using closely related Wikipedia paragraphs, were artificially constructed, which may affect generalizability to diverse real-world data. Second, while our RAG evaluation used a realistic application, it was confined to a single multiple-choice question dataset. Evaluating RAG performance on open-ended tasks, which poses greater challenges in developing robust scoring methods [28], remains an important next step.

Both our method and the LlamaIndex Semantic Splitter operate at the sentence level. However, Max–Min offers greater semantic sensitivity by dynamically assessing sentence relationships, unlike LlamaIndex's fixed-size sentence windows. This allows our method to better handle subtle semantic shifts. Nevertheless, achieving even more granular segmentation, such as splitting sentences at internal semantic change points (e.g., parenthetical notes), would require further investigation into lower-level text analysis techniques. Future research should focus on validating the Max–Min chunker across a broader range of datasets and RAG applications. Additionally, exploring its applicability to other NLP tasks, such as text summarization and document classification, presents a promising avenue for extending its utility.

Acknowledgements

We would like to thank Béla Bálint for scraping Wikipedia and creating the datasets used for dynamic classification.

Author contributions

PSz initiated the research. The method was developed by M.N. and Cs.K., Cs.K. implemented the algorithm and conducted its evaluation under the supervision of M.N. and PSz.. Cs.K. and M.N. wrote the manuscript, and P.Sz. reviewed it.

Funding

Open access funding provided by Budapest University of Technology and Economics. The research presented in this paper, carried out at BME, was supported by the European Union project RRF—2.3.1–21–2022–00004 within the framework of the Artificial Intelligence National Laboratory Grant Nr. TKP2021-NVA-02. Csaba Kiss has been also supported by the Cooperative Doctoral Programme of the National Research, Development, and Innovation Fund.

Data availability

Synthetic clustering data made by us from Wikipedia paragraphs is available at [this OneDrive folder](#). 3GPP documents are available at <https://www.3gpp.org/specifications-technologies/specifications-by-series>

Declarations

Competing interests

We declare that the authors have no Conflict of interest as defined by Discover, or other interests that might be perceived to influence the results and/or discussion reported in this paper. The authors declare no Conflict of interest.

Received: 19 February 2025 / Accepted: 30 May 2025

Published online: 13 June 2025

References

1. Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, Küttler H, Lewis M, Yih W-T, Rocktäschel T, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Adv Neural Inform Process Syst*. 2020;33:9459–74.
2. Francis S, Moens M-F. Investigating better context representations for generative question answering. *Inform Retr J*. 2023;26(1):7.
3. Wadhwa H, Seetharaman R, Aggarwal S, Ghosh R, Basu S, Srinivasan S, Zhao W, Chaudhari S, Aghazadeh E. From RAGs to rich parameters: probing how language models utilize external knowledge over parametric information for factual queries. 2024.
4. Shi F, Chen X, Misra K, Scales N, Dohan D, Chi EH, Schärli N, Zhou D. Large language models can be easily distracted by irrelevant context. In: *International Conference on Machine Learning*. PMLR; 2023. pp. 31210–27.
5. Qian H, Liu Z, Mao K, Zhou Y, Dou Z. Grounding language model with chunking-free in-context retrieval. [arxiv:2402.09760](#) [Preprint]. 2024.
6. LlamaIndex: Sentence splitter (Parse text with a preference for complete sentences.). 2024. https://docs.llamaindex.ai/en/stable/api_reference/node_parsers/sentence_splitter/. Accessed 4 Oct 2024.
7. LlamaIndex: Semantic Nodeparser (Splits a document into Nodes, with each node being a group of semantically related sentences.). 2024. https://docs.llamaindex.ai/en/v0.10.17/api/llama_index.core.node_parser.SemanticSplitterNodeParser.html. Accessed 4 Oct 2024.
8. Jimeno-Yepes A, You Y, Milczek J, Laverde S, Li R. Financial report chunking for effective retrieval augmented generation. *ArXiv abs/2402.05131*. 2024.
9. Juvekar K, Purwar A. Introducing a new hyper-parameter for RAG: context window utilization. [arxiv:2407.19794](#) [Preprint]. 2024.
10. Ram O, Levine Y, Dalmedigos I, Muhlgay D, Shashua A, Leyton-Brown K, Shoham Y. In-context retrieval-augmented language models. *Trans Assoc Comput Linguist*. 2023;11:1316–31. https://doi.org/10.1162/tacl_a_00605.
11. Guu K, Lee K, Tung Z, Pasupat P, Chang M-W. REALM: retrieval-augmented language model pre-training. [arxiv:2002.08909](#) [Preprint]. 2020.
12. Schwaber-Cohen R. Chunking strategies for LLM applications. 2023. <https://www.pinecone.io/learn/chunking-strategies/>. Accessed 9 Jan 2025.
13. Izacard G, Lewis P, Lomeli M, Hosseini L, Petroni F, Schick T, Dwivedi-Yu J, Joulin A, Riedel S, Grave E. Atlas: few-shot learning with retrieval augmented language models. *J Mach Learn Res*. 2023;24(251):1–43.
14. Zhou X, Li G, Liu Z. LLM as DBA. 2023.
15. Kamradt G. Semantic chunking. 2024. <https://www.youtube.com/watch?v=8OJC21T2SL4&t=1933s>. Accessed 9 Jan 2025.
16. LangChain: Semantic Chunker. 2024. https://api.python.langchain.com/en/latest/text_splitter/langchain_experimental_text_splitter.SemanticChunker.html. Accessed 9 Jan 2025.
17. Qu R, Tu R, Bao F. Is semantic chunking worth the computational cost?. 2024.
18. Liu Z, Simon C-E, Caspani F. Passage segmentation of documents for extractive question answering. 2025.
19. Pesi RD, Mathew JG, Mecella M, Aiello M. Advanced system integration: analyzing OpenAPI chunking for retrieval-augmented generation. [arxiv:2411.19804](#) [Preprint]. 2024.
20. Xiao S, Liu Z, Zhang P, Muennighoff N. C-pack: packaged resources to advance general Chinese embedding. 2023.
21. Khattab O, Zaharia M. Colbert: efficient and effective passage search via contextualized late interaction over bert. In: *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2020. pp. 39–48.
22. Nikbakht R, Benzaghta M, Geraci G. TSpec-LLM: an open-source dataset for LLM understanding of 3GPP specifications. 2024.
23. Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay E. Scikit-learn: machine learning in python. *J Mach Learn Res*. 2011;12:2825–30.

24. Vinh N, Epps J, Bailey J. Information theoretic measures for clusterings comparison: variants, properties, normalization and correction for chance. 2009;18.
25. Rosenberg A, Hirschberg J. V-measure: a conditional entropy-based external cluster evaluation measure. In: Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL). 2007. pp. 410–20.
26. Steinley D. Properties of the hubert-arabie adjusted rand index. *Psychol Methods*. 2004;9:386–96. <https://doi.org/10.1037/1082-989X.9.3.386>.
27. Singer S, Nelder J. Nelder-mead algorithm. *Scholarpedia*. 2009;4(7):2928.
28. Roychowdhury S, Soman S, Ranjani HG, Gunda N, Chhabra V, Bala SK. Evaluation of RAG metrics for question answering in the telecom domain. [arxiv:2407.12873](https://arxiv.org/abs/2407.12873) [Preprint]. 2024.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.